Lumina AI Tutor: Presentation Script and Interview Q&A

Nov 1, 2026 · @Adam

A word-for-word English script for your 10-minute session, followed by 60 likely questions with answers. Every fact was checked against your code and your evaluation results.

Before the session: six things to check

Any one of these going wrong becomes the interviewer's first follow-up.

1. **Fix the README.** It mentions LangGraph, LangChain, Pinecone and Colab, but none of them is in the code. The backend uses its own state machine, vectors stored in MongoDB and in-process cosine search. An interviewer who opens GitHub will see the mismatch, so correct the README first.
2. **Warm up the demo site.** The free SnapDeploy container can be idle and slow. Open the site ten minutes before, log in, and check `/health`.
3. **Upload the demo PDF in advance.** The Slide 8 question "Where does the Calvin cycle happen?" only works if a PDF containing that topic is already uploaded. Keep a screenshot of a good answer as a backup.
4. **Know what "78/79" means.** The one failing case is `near-01`, a related but unanswerable question. The scripted LLM always answers when a sentence overlaps, so that case needs a real model to pass.
5. **Know that "100% off-topic refused" is in-sample.** It is 20 of 20 on the set used to write the patterns. The honest estimate is the held-out check: 21 of 25 unseen prompts caught (84%). Quote both.
6. **Describe embeddings accurately.** The evaluation used deterministic hash (lexical) embeddings. The code also supports Google `gemini-embedding-001`, but your status notes say it was not live-tested. Say "tested" only for what you ran yourself on the deployed app.

Speaker script (about 10 minutes)

Read it aloud once or twice and then say it in your own words. Slides 9 and 10 stay closed unless the interviewer asks.

Slide 1: Title (20 seconds)

"Hi, I'm Afzal. Lumina is a study tutor that answers from the learner's own material, stays on topic, and can show how every answer was made. I'll cover the problem, the approach, the architecture, and then a live demo."

Slide 2: The problem (45 seconds)

"A study tool has to be correct, on-topic and accountable. A generic chatbot is none of the three. First, it is ungrounded: it ignores the student's notes and shows no sources. Second, it is off-topic: it answers anything and wastes tokens and focus. Third, it is unbounded: nothing limits cost, tools or prompt injection coming from uploaded files."

Slide 3: Approach (60 seconds)

"I turned that into four principles. Ground: answers come from the learner's own PDFs with page citations. Verify: the system checks that the answer is supported before replying. Bound: there are hard limits on steps, tools, tokens and time. Observe: every step is traced and I measure the system with an evaluation harness. These came from studying RAG pipelines, state-machine orchestration, guardrail patterns and LLM evaluation metrics."

Slide 4: Five methods (60 seconds)

"I compared five designs. A plain chatbot is neither grounded nor controlled. Prompt-only guardrails can be bypassed. Basic RAG is grounded but not controlled, so I used it as a building block. A free-form autonomous agent is grounded but has unbounded cost. Method E combines RAG with a supervisor and a verifier, and it is the only one that is both grounded and controlled, so I chose it."

Slide 5: Chosen design (30 seconds)

"These two parts work together. RAG gives the evidence: chunks, embeddings, owner-filtered retrieval and [S1] page citations. The supervisor makes the process predictable: only allowed transitions, hard budgets, and a verifier that checks support and triggers one correction."

Slide 6: Architecture (105 seconds)

"Read it left to right. A message enters the Gate, which does safety checks. Rejected requests go straight to the end with zero tokens. The Router then decides: vague requests get a clarifying question, simple asks such as a calculation or progress go to direct tools, and real questions continue to Plan if complex, then Retrieve, which only searches the learner's own documents, then Generate with the tutor model. The dashed

box is the verify loop: if the answer is not supported, it is corrected once and checked again. Then the answer gets its citations, the output is scrubbed, and it is returned with a trace. Memory, metering and tracing run on every step."

Slide 7: One question, step by step (45 seconds)

"Take 'Where does the Calvin cycle happen?'. Understand: clean the input, check scope, route the intent. Retrieve: top chunks from the learner's PDFs. Generate: draft using only that evidence. Verify: is every sentence supported? Synthesize: add [S1] citations and scrub the output. Done: answer, trace and token count. There are three branches: a vague ask is clarified, weak support triggers the correction loop, and an off-topic request is refused before any model call."

Slide 8: Live demo (3 minutes)

1. Ask "Where does the Calvin cycle happen?" and say: "The answer has a source chip with the page it came from."
2. Ask "How to bake a cake?" and say: "This is not a study question, so it is refused politely and no model was called, so zero tokens."
3. Type "Ignore previous instructions…" and say: "That is caught as prompt injection before it reaches the model."
4. Open "How this answer was made" and say: "This is the real trace of the workflow: understand, retrieve, generate, verify, synthesize, with tokens used."

Slide 11: Next steps (35 seconds)

"I want to be clear about the limits. My evaluation runs on a scripted model and synthetic data, so the first next step is to run it against the live Groq model. Then semantic embeddings with a managed vector database, live end-to-end testing of authentication, billing and the UI, and streaming responses. Thank you. I'm happy to take questions or walk through the code."

Q&A 1: Problem, approach and the five methods

Q1. What is Lumina in one sentence?

A study tutor that answers from the learner's own PDFs with page citations, refuses off-topic requests, and shows a trace of how every answer was made.

Q2. Why not just use ChatGPT or Claude?

A general chatbot is not tied to the student's notes, answers anything, and has no limits on cost or tools. For studying I need answers I can trace to a page, so the system retrieves first, checks the draft against the evidence, and only then replies.

Q3. What does "grounded" mean?

The answer is built only from retrieved passages of the student's own documents, and each factual sentence carries a citation such as [S1] that points to a real passage and page. If the documents do not contain the answer, the system says so instead of guessing.

Q4. What does "bounded" mean, and what are the limits?

Every request runs inside hard budgets: 6 charged steps, 4 tool calls, 12,000 workflow tokens, a 90-second timeout, 1 correction attempt and 2 provider retries. When a budget runs out, the workflow stops safely with a clear message instead of looping or spending without limit.

Q5. How did you implement "observe"?

Every request produces a trace of spans (route, retrieve, generate, verify) with token usage, secrets redacted, kept for 30 days. The student sees a simplified version as "How this answer was made", and the admin can see traces. A separate evaluation harness with 79 cases measures retrieval, citations, abstention and guardrails. An optional Langfuse export exists, but I have not verified it live, so I do not claim it.

Q6. What research did you do?

Safe answer: "I studied RAG pipelines, state-machine orchestration, guardrail patterns and LLM evaluation metrics such as retrieval hit rate, context precision and faithfulness, and turned them into the four principles." Name a specific paper only if you have actually read it.

Q7. What is new here compared with a normal RAG chatbot?

It combines three things that are usually built separately: private-document RAG, a controlled agent workflow with a verifier, and per-user cost metering with security. The contribution is the reliable process around the model, not a new model.

Q8. Why did you reject prompt-only guardrails?

A prompt is a request to the model, not a guarantee, and it can be bypassed by rephrasing or injected text. I still use a tutor prompt as one layer, but the rules that matter are enforced in code: input checks, a scope guard, a tool allow-list and an output scrub.

Q9. Why not a free-form autonomous agent?

It can loop, call tools in an unplanned order and spend tokens without a ceiling, and it is hard to test. In a paid product that means unbounded cost and security risk. My supervisor only allows listed transitions and fixed budgets.

Q10. Why not LangGraph? The README mentions it.

The workflow is small enough to be one explicit transitions table of about 11 states that I can unit-test and enforce budgets on directly, with no extra dependency. The trade-off is that I do not get a framework's ecosystem or visual tooling. If the README says LangGraph, say plainly that the README is wrong and the code does not use it.

Q11. Why is basic RAG only a building block?

Basic RAG gives grounding but no control: it retrieves and answers even when the evidence is weak, and nothing checks the draft. I kept its retrieval part and added routing, a verifier and budgets around it.

Q12. What is the verifier? Is it another LLM?

No, it is deterministic code, not a second model. It checks that every cited id exists in the retrieved sources, scores how well each sentence is supported by the evidence (threshold 0.60), checks numbers, and flags structure problems. If it fails, the tutor gets one correction attempt with specific feedback and is verified again.

Q13. What is the verifier's weakness?

It is lexical, so it is a heuristic and not proof of factual correctness. A correct answer that is heavily paraphrased can score low and be marked unsupported. Moving to semantic or entailment-based checking is a next step. Say this yourself before the interviewer finds it.

Q14. Why chunk size 1000 and overlap 150?

They are sensible defaults that keep a passage self-contained while the overlap stops sentences being cut at a boundary. They are configuration values, and I did not run a systematic tuning study, so I do not claim they are optimal.

Q15. Which LLM do you use?

Groq's `openai/gpt-oss-120b`, pinned in settings with no fallback model, so behaviour and cost stay predictable. Embeddings come from Google `gemini-embedding-001` when configured.

Q&A 2: Architecture, flow and live demo

Q16. Explain the architecture in 30 seconds.

A message goes through a safety Gate, then a Router. Simple asks go to deterministic tools, vague asks are sent back as a clarification question, and real questions continue to Retrieve, Generate and Verify. A failed check loops to Correct once. Then Synthesize adds citations and scrubs the output, and Done returns the answer with its trace. Memory, metering and tracing wrap every step.

Q17. What does the Gate check?

Input validation (length, malformed text), a prompt-injection check, and a scope guard. The scope guard uses pattern rules for 8 off-topic categories in English and Hinglish. Only when a request is ambiguous does a small LLM classifier decide, and it must be at least 75% confident to refuse. Refusals from the pattern layer happen before any model call.

Q18. How does the Router decide?

Deterministic rules first (keywords and patterns) across 10 intents: chat, doc_qa, lesson, quiz, evaluate, progress, calculate, study_plan, clarify and off_topic. Only if the rules are unsure does it spend a small LLM call to classify. It is cheap and predictable by default.

Q19. When does the Planner run?

Only for compound requests such as a study plan. It produces a short bounded plan that is validated against the step and tool limits before anything runs. Simple questions skip it.

Q20. Which tools exist and how are they kept safe?

Seven registered tools: search_documents, get_passages, summarize_document, generate_quiz, read_progress, calculate and save_preference. The tutor may use only five of them (an allow-list). save_preference needs explicit user approval and is not available to the tutor at all. Each tool has a permission, a timeout (default 20 seconds), validated arguments, and an audit entry that logs the tool name only, never the arguments.

Q21. When is the "zero tokens" claim true?

When the pattern guard or the injection check refuses, no model is called, so zero tokens are used. If a request is ambiguous and the LLM classifier is used, a small number of tokens is spent, and the UI shows that number honestly. So say "zero for clearly off-topic requests", not "always zero".

Q22. What if verification keeps failing?

There is exactly one correction attempt. If the draft is still not supported, Synthesize removes unsupported content and invalid citations, and the reply carries a limitation note instead of invented content.

Q23. What happens on a timeout or provider error?

A 90-second timeout stops the workflow safely with a clear message. A provider error also returns a safe message and the reserved tokens are released, so the student is not charged for a failed run.

Q24. How does metering work?

Before the workflow, an atomic reserve takes a token estimate from the user's plan. After it, the actual usage is settled and the unused part is released. The limit is enforced on the server, not in the frontend.

Q25. How do you stop one user retrieving another user's data?

Every search is filtered by owner. Document ids sent by the client are never trusted: the server checks that each id is a ready document owned by the logged-in user and drops the ones that fail. A test confirms that a foreign document id returns no evidence.

Q26. What if the interviewer uploads their own PDF?

Great, let them. It accepts PDF, TXT and Markdown up to 10 MB; scanned or encrypted PDFs are rejected. If the answer comes back as "not enough evidence", say: "That is the abstention feature. The system does not guess when no passage supports the answer."

Q27. How was the prompt injection caught?

The input check looks for instruction-override patterns and refuses before any model call. For text hidden inside an uploaded document, passages that contain instruction-like text are excluded from the evidence and the answer carries a note. Source text is also wrapped and labelled as data, never as instructions.

Q28. Is "How this answer was made" real or just UI?

It is real. The supervisor records every state it visits (understand, retrieve, generate, verify, synthesize) with its status, and the UI shows that list with tool calls, LLM calls and tokens used. It is not generated text.

Q29. What do I see when I click a citation?

The source filename, the page number and the exact passage that supported the claim. A citation id that is not in the retrieved set is removed by the verifier and synthesis, so a fabricated citation cannot reach the student.

Q30. Can I break it?

Be honest: in a held-out check of 25 new off-topic prompts, 4 got through (84% caught). Not covered: other languages, indirect or disguised requests, and long messages mixing study and non-study content. Saying this first removes the gotcha.

Q31. Why is the demo slow?

A grounded answer takes several steps (retrieve, generate, verify) and calls an external LLM, and the free-tier container can be cold. Streaming responses is a listed next step.

Q32. How are login and user data protected?

Passwords use Argon2id. The access token is a 15-minute JWT kept in memory only, and the refresh token is a rotating httpOnly SameSite=Strict cookie stored hashed; reusing an old refresh token revokes all sessions. State-changing requests need a custom CSRF header.

Q&A 3: Results and code

State the caveat before they ask: these numbers come from a regression harness, not a real-world accuracy study.

Q33. What does "78/79 test cases pass" mean?

The evaluation dataset has 79 cases across 12 categories (answerable, unanswerable, ambiguous, malformed, injection, scope and others), and 78 pass. The failing one is `near-01`, a related but unanswerable question, because the scripted LLM always answers when any sentence overlaps. That outcome depends on a real LLM's judgement, so it is not a property of the product.

Q34. Is the 100% retrieval hit rate meaningful?

It is real for this dataset, but the dataset is small, synthetic and labelled by me, and the embeddings are a deterministic lexical hash, so overlap with the query is easy. It shows the pipeline works and protects against regressions. It does not prove real-world accuracy, and the slide says so.

Q35. Why a scripted LLM for evaluation?

So the run is deterministic, free and repeatable. It tests my own logic (routing, guardrails, retrieval filtering, verifier, synthesis and budgets), not the model's quality. Model quality needs a live evaluation, which is my first next step.

Q36. Which is true, 100% off-topic refused or 84%?

Both, on different sets. 20 of 20 is on the labelled set I used to write the patterns, so it is in-sample. The held-out check on 25 unseen off-topic prompts caught 21 (84%), and 1 of 25 study questions was wrongly refused (4%). Those misses were fixed afterwards. The held-out number is the honest estimate.

Q37. What do context precision 0.90 and recall 1.0 mean?

Precision 0.90 means about 90% of the retrieved passages were relevant. Recall 1.0 means the passages needed to answer were all retrieved. Both are measured on the synthetic set.

Q38. How did you measure "0 secret leaks"?

The evaluation and tests check that configured secret values (API keys and similar) never appear in answers, traces or errors, and the output scrub redacts them. It covers the cases I tested, so say "no leaks in my tests", not "it cannot leak".

Q39. How many tests do you have?

On my last run the backend had 174 passing pytest tests and the frontend 6 passing Vitest tests, using a fake MongoDB and mocked providers. Run them again on your own machine before the interview and quote the number you see.

Q40. How would you measure real accuracy?

Run the same harness against the live Groq model and semantic embeddings, add a human-reviewed question set from real course PDFs, and track retrieval hit rate, citation correctness, faithfulness and abstention. Also track how often the verifier wrongly rejects correct paraphrased answers.

Q41. What does the TRANSITIONS table do?

It lists, for every state, which states may come next. The supervisor calls `go(next)`, and a move that is not in the table raises an `IllegalTransition` error. So the flow cannot skip verification or jump back to a finished state, and the rule is enforced in code, not by a prompt. The slide shows a shortened version; the real table also covers START, UNDERSTAND, CLARIFY, PLAN, RETRIEVE and FAILED.

Q42. What are "charged" steps?

PLAN, RETRIEVE, GENERATE, VERIFY and CORRECT use up the step budget of 6. UNDERSTAND, CLARIFY and SYNTHESIZE are free bookkeeping. If the budget is exhausted, a move to a charged state is refused and the workflow ends safely.

Q43. How is the timeout implemented?

The whole workflow runs inside `asyncio.wait_for` with `workflow_timeout_seconds` (90). On timeout the student gets a short "stopped safely" message, and the metering step releases the reservation.

Q44. How is the tool allow-list enforced?

The supervisor builds a tool context with the allowed tool names (5 for the tutor). The registry rejects any tool not in that set, any call over the 4-call budget, and any arguments that fail schema validation. The evaluation includes an invented tool name and bad arguments, and both are rejected.

Q45. Why a support threshold of 0.60?

It is a hand-chosen cut-off: high enough to catch sentences with little overlap with the sources, low enough to allow reasonable paraphrase. I did not run a systematic threshold sweep, and I say that plainly. A semantic check is the improvement.

Q46. How does the verifier compute support?

For each sentence it normalises and stems the words, drops citation markers and digits, and computes the share of its content words that also appear in the cited passages. A sentence below 0.60 is flagged unsupported. Numbers in a sentence must appear in the evidence, otherwise it is flagged too. Questions, headings and very short fragments (under 4 content words) are skipped. It is a bag-of-words check, so it cannot tell a true paraphrase from a false one.

Q47. What if the model returns invalid JSON?

All model outputs are validated against a Pydantic schema. On failure the system asks the model to correct its output, up to 2 repair attempts. The client also retries on 429 or 5xx errors with backoff, uses only the one pinned model, and never reads or shows the model's reasoning text.

Q&A 4: Stack, deployment, billing, scale and Chat with PDF

Q48. What are your next steps, and why in that order?

First, run the evaluation against the live Groq model, because every number I have comes from a mock. Second, use semantic embeddings and a managed vector database for quality and scale. Third, live-test authentication, billing and the UI end to end. Fourth, add streaming for a better feel. The order follows risk: unproven quality first, polish last.

Q49. How does vector search work today, and will it scale?

Chunk vectors are stored in MongoDB and scored with cosine similarity in the application process, filtered by owner and selected documents. That is fine for a student's few documents, but it scans every candidate, so at larger scale I would move to a vector index such as Atlas Vector Search or a managed vector database.

Q50. What if the embedding model changes?

The retriever compares the stored vectors' dimension with the query embedding and re-embeds a document's chunks when they differ, so two embedding models are never silently mixed. Call this "live-tested" only if you have seen it work on Atlas yourself.

Q51. What is your tech stack and why?

FastAPI with Pydantic v2 for async APIs and strict validation, MongoDB (Atlas free tier for deployment) for flexible documents, React 18 with TypeScript and Vite for the UI, and Groq for fast inference of an open-weight model. The choices were driven by validation, cost and speed to ship.

Q52. How is it deployed?

A multi-stage Dockerfile builds the React app, then a slim Python image serves both the API and the static frontend from one origin, running as a non-root user. It runs as a free container on SnapDeploy with MongoDB Atlas M0, and exposes `/health` and `/ready` endpoints.

Q53. How does billing work, and was it tested live?

Plans are Free (250k tokens, 3 documents), Pro at ₹499 and Premium at ₹1499, each for 30 days. The price is decided on the server, Razorpay's signature is verified with HMAC, and webhooks are idempotent through a unique event key. The logic and tests exist with a mocked Razorpay; a live payment has not been tested, and you should say exactly that.

Q54. How does the new Chat with PDF feature work?

In Tutor chat the student attaches a PDF, and the conversation saves its document scope. Each follow-up reloads recent history and retrieves fresh passages from only those documents, adding the previous question to the search text so a follow-up like "explain that again" still finds the right passage. The server re-checks document ownership on every turn, and a new chat does not inherit the old scope.

Q55. How do you handle rate limiting and abuse?

Failed logins are throttled per email and IP in MongoDB with automatic expiry, and plan token limits are enforced on the server. The general API limiter is a sliding window held inside one server process, so with several instances it would not be shared. Distributed

rate limiting is listed as not implemented.

Q56. What happens to my PDF? Is it private?

Text is extracted, chunked and stored under your account only, and other users cannot retrieve it. Traces redact secrets and are removed after 30 days. The optional Langfuse export would send metadata only, not document text, but I have not verified that live.

Q57. What was your biggest challenge?

A true answer: making the verifier strict enough to block unsupported claims without rejecting correct paraphrases, which is why a lexical check is a known limit. A recent real example: long answers were being cut off by a low output-token limit and the provider returned HTTP 400, so I raised the limit and added a retry. Tell this story only after you have confirmed on the deployed app that the error is gone.

Q58. What is special about the frontend?

A dark "Observatory" theme where yellow always means evidence, a Ctrl+K command palette, 15 routes, an access token held only in memory, a keyboard-friendly chat, and motion that respects the user's reduced-motion setting.

Q59. What would you change for 10,000 users?

Move vectors to a managed index, make document ingestion asynchronous with a queue, share rate limits through a cache such as Redis, stream responses, and add real monitoring. I would also load-test first, because I have not.

Q60. What features does it not have?

No web search, no email or OTP, no password reset, no MFA, no streaming responses, no account export or deletion, and no distributed rate limiting. It is better to list these yourself than to be caught claiming them.

Claims to avoid, and what to say instead

The left column does not match your code or your evidence. An honest limit sounds more mature than an overclaim.

Do not say	Say instead
"It uses LangGraph, LangChain or Pinecone."	"It uses a custom state machine, vectors stored in MongoDB and in-process cosine search. A managed vector database is a next step."

Do not say	Say instead
"Accuracy is 100%."	"On a synthetic, author-labelled set the regression harness passes 78 of 79. Real accuracy needs a live evaluation."
"It blocks 100% of off-topic requests."	"It caught 84% on a held-out set. Known gaps: other languages and disguised requests."
"Refusals always use zero tokens."	"Clear off-topic and injection requests use zero. Ambiguous ones need a small classifier call."
"The verifier proves the answer is correct."	"The verifier is a lexical support check, not proof. It can wrongly flag a paraphrase."
"Billing was tested live."	"The logic and tests use a mocked Razorpay. A live payment has not been tested."
"Tracing runs on Langfuse."	"I built my own tracing with spans, redaction and 30-day retention. The Langfuse export is optional and not verified live."
"It is production-scale."	"It is a single-instance, free-tier deployment. Scale needs a vector index, a queue and a shared rate limiter."
"It streams responses."	"Not yet. Streaming is a next step."
"It has password reset, email OTP or MFA."	"Not yet."

When you do not know an answer

1. Say: "I have not measured that yet."
2. Explain how you would check it: which test or metric.
3. Return to something you have measured.

One last tip: state your limits yourself, on slides 9 and 11, before anyone asks. A weakness you name reads as maturity; one the interviewer has to find reads as a gap.